Smart Health 33 (2024) 100498



Contents lists available at ScienceDirect

Smart Health

journal homepage: www.elsevier.com/locate/smhl





Ultra low-power, wearable, accelerated shallow-learning fall detection for elderly at-risk persons

Jingxiao Tian [a], Patrick Mercier [b], Christopher Paolini [a,*]

[a] Electrical and Computer Engineering Department at San Diego State University, 5500 Campanile Drive, San Diego, 92182, CA, USA
[b] Electrical and Computer Engineering Department at University of California, San Diego, 9500 Gilman Drive, La Jolla, 92093, CA, USA

ARTICLE INFO

Keywords:
Caffe
CNN
Fall detection
FPGA
Edge computing
Wearable technology
Elderly care
Accelerometer
Gyroscope
Shallow neural network

ABSTRACT

This work focuses on the development and manufacturing of a wireless, wearable, low-power fall detection sensor (FDS) designed to predict and detect falls in elderly at-risk individuals. Unintentional falls are a significant risk in this demographic, often resulting from diminished physical capabilities such as reduced hand grip strength and complications from conditions like arthritis, vertigo, and neuromuscular issues. To address this, we utilize advanced low-power field-programmable gate arrays (FPGAs) to implement a fixed-function neural network capable of categorizing activities of daily life (ADLs), including the detection of falls. This system employs a Convolutional Neural Network (CNN) model, trained and validated using the Caffe deep learning framework with data collected from human subjects experiments. This system integrates an ST Microelectronics LSM6DSOX inertial measurement unit (IMU) sensor, embedded with an ultra-low-power Lattice iCE40UP FPGA, which samples and stores joint acceleration and orientation rate. Additionally, we have acquired and published a dataset of 3D accelerometer and gyroscope measurements from predefined ADLs and falls, using volunteer human subjects. This innovative approach aims to enhance the safety and well-being of older adults by providing timely and accurate fall detection and prediction.

In this paper, we present an innovative approach to utilizing a compact Convolutional Neural Network (CNN) core for accelerating convolutional operations on a machine learning model, suitable for deployment on an ultra-low power FPGA.

1. Introduction

The aging global population has underscored the importance of healthcare skills and ambulatory monitoring for elderly care, evolving into a key focus of interdisciplinary research. As outlined by Torti et al. (2018), technological advancements have made mobile and wireless devices a ubiquitous part of everyday life. Activities of daily living (ADL), such as standing, sitting, lying down, walking, and navigating stairs, are fundamental to daily routines. The ability to recognize, interpret, and monitor these ADLs, along with fall detection, is vital for the effectiveness of context-aware systems (Musci, De Martini, Blago, Facchinetti, & Piastra, 2021; Waheed, Afzal, & Mehmood, 2021). Falls occur when an individual's center of gravity shifts, leading to a loss of equilibrium. The World Health Organization (WHO) defines a fall as an unintentional, unexpected, or uncontrollable event that results in the person coming to rest on the ground or another lower level (Ramachandran & Karuppiah, 2020), (Chander et al., 2020).

Our team has developed and released a dataset featuring 3D accelerometer and gyroscope readings, which were gathered from pre-defined Activities of Daily Life (ADLs) and fall scenarios using human volunteers. Unlike publicly accessible fall datasets (Casilari,

* Corresponding author.
E-mail addresses: paolini@engineering.sdsu.edu, cpaolini@sdsu.edu (C. Paolini).

https://doi.org/10.1016/j.smhl.2024.100498
Received 28 January 2024; Received in revised form 29 March 2024; Accepted 31 May 2024
Available online 5 June 2024
2352-6483/© 2024 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

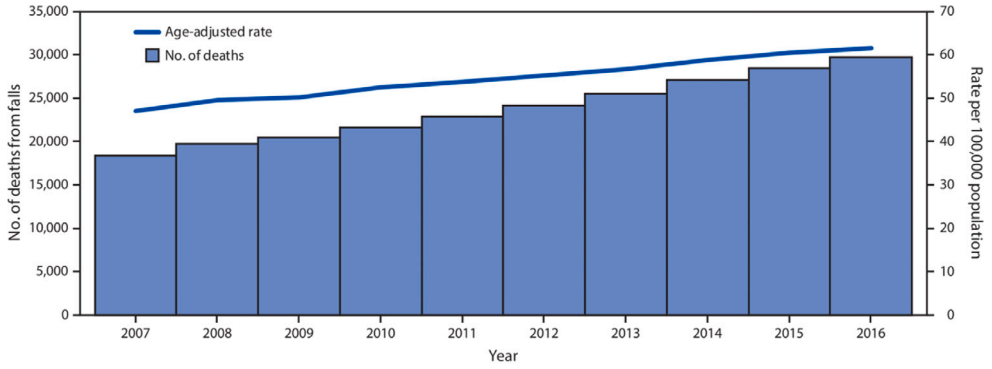


Fig. 1. The number of fall-related deaths and rates among adults over 65 in the United States from 2007 to 2016 (Burns & Kakara, 2018a).

Santoyo-Ramón, & Cano-García, 2017), which typically provide data from sensors located on the wrist, chest, and waist, our dataset uniquely includes readings from the shinbone area. We employed a recurrent neural network with Long Short-Term Memory (LSTM) architecture, training it with data on transient orientation and acceleration of body parts leading up to a fall. Our analysis, using receiver operating characteristic curves, demonstrated that the shinbone sensor location provides the highest area under the curve (AUC) for fall detection (Paolini, Soselia, Baweja, & Sarkar, 2019).

While it is possible to mitigate certain external risk factors for falls through careful measures, internal risk factors present a more complex challenge, and it is not always feasible to completely prevent falls in every situation. Given the severe and often expensive consequences of falls, it is crucial to have reliable and prompt fall detection. This necessitates the development of real-time fall detection algorithms that are not only accurate but also resilient and robust in their performance (Santiago, Cotto, Jaimes, & Vergara-Laurens, 2017).

Maglogiannis, Ioannou, and Tsanakas (2016) have highlighted the considerable benefits of using wearable motion sensors to monitor individuals at risk of falling. Each sensor provides a 1D signal, the combination and intelligent processing of data from multiple sensors aligned along different axes enable these devices to effectively determine 3D motion. Unlike optical motion detection systems, inertial sensors do not require a line of sight and can function effectively within or behind objects without issues of overlap (Boutellaa, Kerdjidj, & Ghanem, 2019; Casilari, Alvarez-Marco, & García-Lagos, 2020). These wearable devices are advantageous due to their lightweight, comfortable design and ease of use, enabling users to move freely both indoors and outdoors without being restricted to a specific area. Wearable devices are advantageous as they require much less infrastructure and are more cost-effective than smart environments, which are technologically enhanced spaces using interconnected sensors and devices for automation, efficiency, and improved user interaction and experience while preserving the user's privacy. Additionally, unlike acoustic sensors, wearable sensors are not affected by environmental noise. Thus, they are highly effective for developing automatic fall detection systems (Boutellaa et al., 2019; Casilari et al., 2020). While there are commercial devices and patents for fall detection systems, they often fall short of expectations. Key issues include frequent false alarms, high costs for setup and maintenance, and lack of ergonomic design.

Deaths resulting from falls among people over 65 years of age in the United States from 2007 to 2016 increased by 31% (3.0% annually). In 2016, there were 29,668 (61.6 per 100,000) deaths nationwide among Americans over 65 who passed away by falls. The rates varied by state, from 24.4 in Alabama to 142.7 in Wisconsin. In 2016, there were almost 30,000 deaths from falls in individuals over 65, with mortality rates varying by state from 24.4 per 100,000 in Alabama to 142.7 in Wisconsin. In 30 states and the District of Columbia, the number of older individuals dying from falls climbed consistently between 2007 and 2016, as shown in Fig. 1 (Burns & Kakara, 2018b). By screening for fall risk and taking action to address modifiable risk factors like polypharmacy or gait, strength, and balance issues, the rising number of deaths from falls in this age group can be reduced as the number of people over 65 increases in the United States. Multiple risk factor-focused interventions that can be started during yearly wellness visits can lower the rate of falls. Healthcare providers can be helped by programs like the CDC's STEADI (Stopping Elderly Accidents, Deaths, and Injuries) in determining fall risk, educating patients, and choosing interventions.

The consequences of a prolonged laying posture can be reduced by adopting automatic fall detection or personal emergency call devices, as well as shortening the time between a fall and reaching health care. An injury or loss of consciousness might prohibit an older person from answering the phone or pushing the emergency button if they falls at home. Furthermore, even though some older individuals do not use their personal emergency call systems (Ojetola, Gaura, & Brusey, 2011)., there is still a significant need for these devices. As a result, over the last decade, various automatic fall detection algorithms have been developed to address this issue.

This study aims to use wearable sensors and embedded machine learning models to reliably and accurately predict falls occurring during Activities of Daily Living (ADL). This paper presents the design of a wireless, wearable, low-power fall detection sensor (FDS) aimed at predicting and detecting falls in elderly individuals at risk. The device is designed to continuously collect data on Activities

of Daily Living (ADL) to evaluate fall risk in users. Those wearing the sensor will receive real-time feedback through mobile device notifications, including behavior modification reminders to minimize fall risk.

The core of this system is the ICE40UP5K FPGA, recognized as the world's smallest FPGA. This iCE40 UltraPlus UP5K FPGA will be programmed with a fixed-function representation of a trained neural network that specializes in fall classification. The neural network model, trained and validated using the Caffe deep learning framework with participant-collected fall data, is an integral part of our device. Our goal is to develop a set of prototype fall sensor devices. These devices will be assembled from a single printed circuit board (PCB) encased in a custom 3D printed housing, equipped with a rechargeable lithium polymer battery and a leg harness. The PCB will incorporate a low-power iCE FPGA from Lattice Semiconductor, alongside a single accelerometer and gyroscope sensor.

We implemented serial communication between an ICE40UP5K FPGA and a STMicroelectronics MEMS IMU sensor using SPI communication to sample acceleration and gyroscope measurements from the sensor and store samples in the FPGA's embedded memory for subsequent input to an embedded neural network classification model. Sequences of acceleration and gyroscope measurement values are characteristic of human falls and human ADLs (Torti et al., 2018; Waheed et al., 2021).

2. Related work

There are various wearable fall detection sensors (FDS) available on the market, commonly worn as pendants around the neck (Mobile help, 2019) or as wristwatches. These devices detect falls and can wirelessly alert a call center to send emergency services if a significant impact is detected. However, companies often include a disclaimer stating, "Fall detection does not detect 100% of falls. If able, the customer should press the help button in an emergency". This help button acts as a failsafe in cases where the device does not detect a fall (a type II error). In situations where a fall leads to unconsciousness, such as a concussion, and the individual is alone, they may be unable to press the emergency button (Gurley, Lum, Sande, Lo, & Katz, 1996). Additionally, even those capable of pressing the button after a fall may sometimes choose not to Porter (2005). These commercial devices predominantly use inertial sensors for fall detection (Diaz, Prado, Roa, Reina-Tosina, & Sanchez, 2004; Hwang, Kang, Jang, & Kim, 2004; Lindemann, Hock, Stuber, Keck, & Becker, 2005; Noury et al., 2003), employing algorithms based on simulated falls by healthy individuals. Despite claims of high detection sensitivity and specificity, often above 90%, these algorithms tend to perform less effectively in real-world scenarios, showing sensitivity and specificity rates of 57.0%±27.3% and 83.0%±30.3%, respectively, when tested with limited real-world fall data (Bagala et al., 2012) from older adults in their natural settings. Our previous research, using IMU sensors at sixteen different body locations to train four machine learning models, indicates that the optimal placement for an FDS is at the front of the shinbone (Paolini et al., 2019; Yhdego et al., 2019). A significant challenge in this field is the scarcity of real-world fall data, which hinders progress in developing more effective fall detection systems.

In the initial phase of our study, we collected data from a cohort of 51 volunteers, ranging in age from 18 to 45 years, encompassing a diverse mix of races and genders. While this group does not directly reflect the primary demographic at risk for falls — the elderly — it provided valuable initial insights into the performance of our fall detection system under varied conditions. This choice was driven by the availability of participants and considerations related to conducting preliminary tests with an emphasis on safety and ethical approval. Acknowledging the limitation of not including older adults, we emphasize the need for subsequent research phases to focus on this demographic, as they represent the most relevant population for fall detection technologies.

To design and implement a mobile, wireless, wearable, low-power medical device that uses a small Field Programmable Gate Array (FPGA), integrated with accelerometer and gyroscope sensors, and that uses Neural Networks (Grossman et al., 2018) to detect when the device wearer has fallen, it is essential to understand the previous work conducted in this field. We must understand what a fall is, why falls happen, and in what age group falls are seen. Finally, we must understand the importance of a Fall Detection System (FDS), effective designs of fall detection systems, which hardware is suitable for implementing FDSs, fall experiments that have been conducted, algorithms used for fall detection, and methods used to detect a fall.

The field of video-based action recognition has witnessed significant advancements, laying a foundational understanding that informs our approach to fall detection. Carreira and Zisserman (2017) introduced a compelling model alongside the Kinetics dataset, setting a new benchmark in action recognition that underscores the importance of comprehensive datasets for training deep learning models. Similarly, the introduction of SlowFast networks by Feichtenhofer, Fan, Malik, and He (2019) presents a novel architecture that effectively captures the dynamic nature of actions, which parallels our efforts to accurately detect falls in real-time. Further, Tian, Yan, Zhai, Guo, and Gao (2022) proposed the Event Adaptive Network (EAN), which adapts to the complexity of actions within videos, a principle that mirrors our adaptive approach to discerning falls from standard movements. Lastly, the work on self-supervised motion representation by Tian, Che, Bao, Zhai, and Gao (2020) illustrates the potential of utilizing local motion cues for action recognition, inspiring our method to leverage subtle yet critical features for fall detection.

Falls have continued to be a primary cause of morbidity and death among elderly people, despite numerous prevention interventions. Serious injuries, such as sustaining a hip fracture and becoming hospitalized, which may lead to death, are common outcomes of falls. Even if there are no significant injuries, fear of falling and the resulting self-limitation of movement and function can lead to nursing home admissions and a loss of individual autonomy, both of which might lead to depression and a negative impact on the victim's quality of life. An aging population is still a major public health issue (Cippitelli, Fioranelli, Gambi, & Spinsante, 2017).

There are certain advantages of using an FPGA to detect falls. Bet, Castro, and Ponti (2019) mention that the system CPU in the majority of existing drop detection systems is a microcontroller or FPGA. In computing, architectural organizations involve the structured design and integration of system components, where microcontrollers enhance intelligence, miniaturization, and

dependability for software solutions and programmable logic like FPGAs are utilized to boost efficiency and processing capacity in hardware designs, thereby optimizing the overall efficiency, reliability, and adaptability of both hardware and software solutions. The use of microcontrollers as processing units can result in better intelligence, downsizing, and dependability, but it is confined to architectural and software applications research (Bet et al., 2019; Ojetola et al., 2011). Furthermore, low-power micro-controllers with limited performance render them unusable for fall detection, and integrating more powerful processors increases power consumption, which is a serious issue for micro-controllers. With the recent development of new low-power FPGA devices such as the Lattice Semiconductor iCE40 UltraPlus, specially developed for embedded AI applications, FPGA technology can provide a versatile and adaptable single-chip architecture that avoids power consumption concerns. Microcontrollers with little processing capability are unable to perform real-time multi-parameter analysis of biological data. Bet et al. (2019) report that FPGAs offer real-time analysis of multidimensional data using statistical methodologies and signal processing algorithms, making them ideal for biomedical patient monitoring devices. Signal processing rates of up to 100 MHz are required in some systems that combine fall detection and EEG signal analysis applications. FPGAs make it possible to transport raw data from accelerometers quickly and efficiently. For fall detection, embedded FPGA memory provides a significant advantage over microcontroller RAM in terms of storing sampled inertial measurement data (Cippitelli et al., 2017; Hussain, Hussain, Ehatisham-ul Haq, & Azam, 2019). Their ability to perform high-speed data processing through parallel operations makes them exceptionally suitable for real-time analysis, crucial in emergency response scenarios. FPGAs are also highly customizable, allowing for tailored hardware logic that optimizes the processing of accelerometer data, enhancing efficiency and accuracy. This customization extends to improved memory utilization, providing larger storage capacity and faster access times for inertial measurement data. Furthermore, FPGAs are known for their lower latency and efficient power consumption, essential features for wearable fall detection devices that require prompt and reliable performance.

3. Methods

3.1. Using convolutional neural networks

Our decision to utilize the iCE40 UltraPlus™ FPGA as our chosen hardware platform was primarily driven by its status as one of the lowest-power hardware capable of executing machine learning models. This attribute aligns with our operational goals, particularly in ensuring energy-efficient processing in real-time scenarios. Our decision to choose low-power apparatus for fall detection systems was guided by the need for them to operate continuously and reliably without using a lot of power. This also means the devices do not need frequent recharging, which is particularly convenient for elderly users.

Incorporating the iCE40 UltraPlus™ FPGA, noted for its minimal energy consumption, demands compliance with specific standards and compatibilities in the machine learning models we deploy. This is where Convolutional Neural Networks (CNN) become an attractive option for our framework (Komeylian, 2021; Komeylian, Paolini, & Sarkar, 2023). The iCE40 UltraPlus™ FPGA is accompanied by a Compact CNN Accelerator IP Core, tailored to the hardware acceleration of key network layers such as convolution, pooling, and batch normalization used in CNN-based deep learning models. By selecting a CNN model, we effectively utilize the inherent synergies of the hardware, ensuring that our system is not just computationally efficient, but also remarkably power-saving.

Our method employs a single 6-axis sensor to capture 1D time-series data, which we then transform into a 2D matrix, or a *picture map*. This approach leverages the data processing strengths of CNN-based models. By converting the data from our 6-axis sensor into a matrix form, we enable the model to more effectively identify and learn patterns associated with fall events. This technique not only aids in recognizing fall-related patterns but also allows the model to independently discern and generalize critical features from the data, irrespective of their explicit interpretability. Through extensive training, our network is fine-tuned to detect and distinguish patterns indicative of falls, effectively differentiating them from non-fall incidents. This method effectively simulates a multi-dimensional sensor environment using data from a single 6-axis sensor, exploiting the robust data processing capabilities of CNNs for accurate fall detection.

3.2. Autonomous CNN feature recognition

In the specific field of fall detection, the significance of features primarily stems from the fundamental physics and biodynamics of human movement. Falls, which can be identified by sudden changes in acceleration and orientation, create unique patterns in sensor data. While these shifts in patterns are measurable empirically, they may not always be easily recognizable or visually discernible, especially when converted into a 2D image matrix for CNN processing.

Our approach heavily depends on the training of the CNN to independently and accurately identify features associated with falls. By extensively training the CNN with datasets that include both real-world fall and non-fall scenarios, the model develops the ability to detect patterns in the sensor data that signify a fall. As a result, the model becomes proficient in recognizing and learning vital features, thereby enabling accurate fall detection.

The cornerstone of our methodology is to independently determine the most crucial features for fall detection. This is achieved through a careful conversion of time-series data into a two-dimensional matrix, followed by thorough training of the CNN. This data-centric strategy leverages the computational strength and learning capacities of deep learning, allowing for the automated identification and integration of significant features from the raw sensor data. Notably, this method does not rely on predefined assumptions about the visual or intuitive characteristics of features that indicate fall incidents.

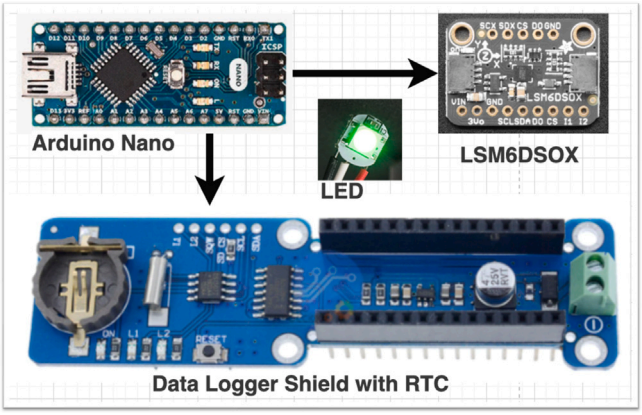


Fig. 2. Fall detection data logger includes an Arduino Nano, LSM6DSOX, and a Data Logger Shield with a Real-time clock (RTC).

4. DATA and MODEL

4.1. Human subjects data collection

To collect and disseminate a dataset, we are using a 6-DoF (Degrees of Freedom) sensor, which includes an accelerometer and gyroscope. This sensor can measure three dimensions of linear acceleration and three dimensions of angular velocity across various rates within a specified range. Our data collection focuses on readings from predefined Activities of Daily Living (ADLs) as well as fall events. And those data are detailed in the work by Tian and Paolini (2023) (Tian & Paolini, 2023). Each participant is equipped with a data capture device attached to their shinbone, featuring the STMicroelectronics LSM6DSOX iNEMO inertial module. This module is set to operate within the $\pm 8\ g$ range (where $1\ g = 9.81\ \mathrm{ms}^{-2}$, representing gravitational acceleration). We systematically collect and document the $x-$, $y-$, and z-axis acceleration data (measured in g) from the module, as well as the yaw, pitch, and roll rate measurements (expressed in degrees per second) provided by the LSM6DSOX.

Prior to participation, all subjects are thoroughly briefed about the experiment, including potential risks and their rights. Written informed consent is obtained from each participant, ensuring they understand and agree to the study's terms, including the possibility of induced falls or stumbles. To mitigate risks, we have implemented stringent safety protocols, including protective gear to prevent severe injury. The experiment is designed and monitored to prioritize participant safety, with continual risk assessments and the option for participants to withdraw at any time without consequence.

During the experiment, each participant wears an Oculus Rift S virtual reality headset, which obscures their view of the actual walking path in our laboratory. Each human participant is immersed in a virtual environment that presents them with a simulation of a straight pathway. Unbeknownst to the participants, as they reach a certain point along a walking path (marked by a cloth on the actual laboratory floor), a Graduate Assistant (the first author of the study) discreetly pulls on the carpet. Those are a part of this study's protocol, approved by our Institutional Review Board (IRB). This action is done with varying degrees of intensity, intended to provoke either a fall or a stumble, with the participant either losing balance and falling or managing to recover and maintain their balance.

It is crucial to acknowledge that our data collection approach, involving systematically induced falls, may introduce a certain level of bias. This is because the dynamics of these induced falls could differ from those occurring naturally, which might affect the reliability and applicability of our collected data.

Additionally, our use of simulated falls represents a deliberate strategy to gather fall data under controlled conditions. However, this approach comes with inherent limitations and potential biases due to the prearranged nature of the falls. In our discussions, we include reflections on these limitations and biases to present a well-rounded and objective perspective on the methodology guiding our research.

Our setup includes a data logger, depicted in Fig. 2, equipped with an LSM6DSOX sensor, which records data at a sampling frequency of 60 Hz. To create a comprehensive dataset of inertial measurement data capturing instances of falls onto a surface, we carried out experiments involving falls with 81 qualified participants. A graphical representation of both fall and no-fall events, as captured by our experiments, is illustrated in Fig. 3.

Fig. 3 illustrates a comparative visualization of accelerometer and gyroscope data captured from the inertial measurement unit (IMU) sensor during both fall and no-fall events. The values presented in this figure represent the magnitude of acceleration (measured in meters per second squared, $\mathrm{m/s}^2$) and angular velocity (measured in degrees per second, °/s) across the three spatial axes (X, Y, and Z). These measurements are plotted over time.

Once the data collection is complete, it requires some pre-processing to format the time series data for compatibility with the input layer of our compact CNN model. This involves converting the raw data into one or more permissible dimensions. For instance, a single 32×32 matrix comprises 1024 data points.

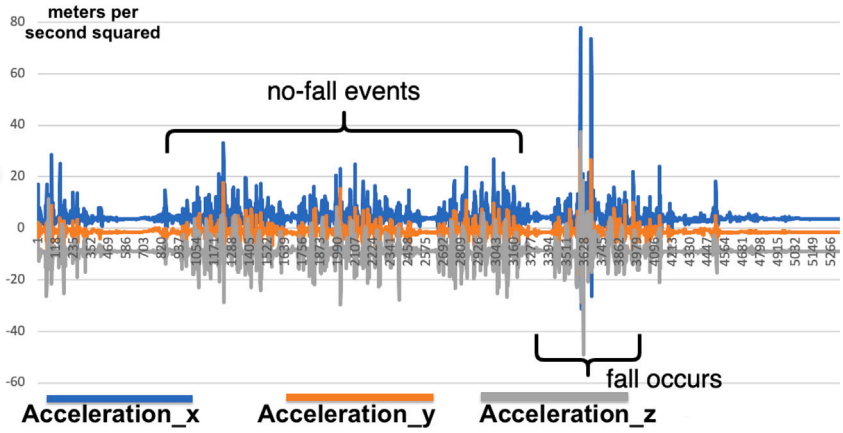


Fig. 3. Data visualization for fall and no-fall events.

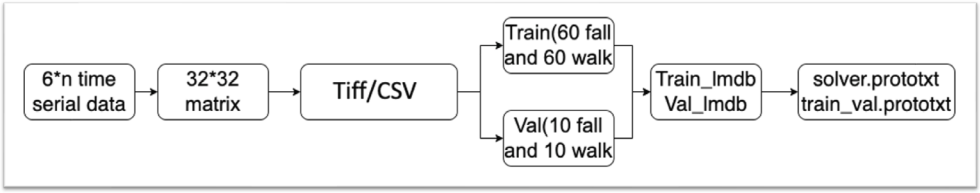


Fig. 4. Fall detection data processing flow chart.

Given that our dataset includes 6-axis data (comprising 3-axis acceleration and 3-axis orientation rate measurements), we can fit $\lfloor 1024/6 \rfloor = 170$ samples into a 32×32 matrix. To achieve this, we utilize 170 data points from the 6-axis data and add four 0-values as padding to complete the 32×32 matrix. As illustrated in Fig. 4, this process involves transforming 1D time series data into a 2D 32×32 image matrix. An example of an image created from a selected 32×32 matrix is displayed in Fig. 5.

In preparing our sensor data for CNN analysis, we adopted a methodical approach to concatenating and reshaping 170 data points from each axis into a 2D matrix. This process was guided by the chronological sequence of data points, employing a sliding window technique to ensure the temporal order is accurately represented in the matrix. By sliding the window one column at a time over a 32 by N matrix, we capture time-sequenced snapshots of the data, thereby creating a continuous and temporally coherent input for the CNN. Through this approach, we ensure that our system can effectively interpret the complex spatial–temporal patterns characteristic of fall events, highlighting the intricate balance between hardware optimization and data integrity in our study.

4.2. Human subjects data processing

In our study, we transform time-series data into a two-dimensional (2D) matrix format, where each matrix element represents a sensor reading. While Fig. 5 might depict the data in a binary black/white visual format, this should be understood as a simplification for illustrative purposes. It is important to recognize that the actual data from the Inertial Measurement Unit (IMU) consists of a range of continuous values, not just binary, capturing detailed patterns that are indicative of falls.

This conversion is primarily motivated by the need to utilize the capabilities of a low-power FPGA platform, specifically the ultra-low-power iCE40 UltraPlus™ FPGA. This platform supports a Compact CNN Accelerator IP Core, which is optimized for accelerating key operations in deep learning models, such as convolution, pooling, and batch normalization.

The choice of a 2D matrix format aligns with the operational requirements of this FPGA platform and the inherent feature of Convolutional Neural Networks (CNNs). CNNs are exceptionally adept at processing spatial data, and by converting our time-series data into a 2D spatial format, we harness this strength. This transformation allows our CNN to effectively identify and learn complex patterns associated with fall events, which are embedded in the IMU data.

The precision of the sensor readings is translated into pixel intensity values within the 2D matrix, ensuring the preservation of the data's original resolution. The visualization of the data is merely for display and does not reflect the true quality and resolution of the data used in our analyses. The actual 2D matrix used for CNN contains continuous, non-binary sensor readings, maintaining the intricate nature of the original data and preventing any analytical oversights.

Furthermore, the sensor data, particularly the acceleration and orientation rate values represented in the 32×32 matrix in IEEE 754 format as FP32 float, undergo additional pre-processing steps. These steps include scaling and normalization to positive values, followed by quantization to INT8. This procedure is not only crucial for maintaining data precision but also ensures that the matrix, post-transformation, retains the necessary quality and consistency for effective CNN analysis.



Fig. 5. A selected post-processed 32 × 32 pixel image generated from time series fall data.

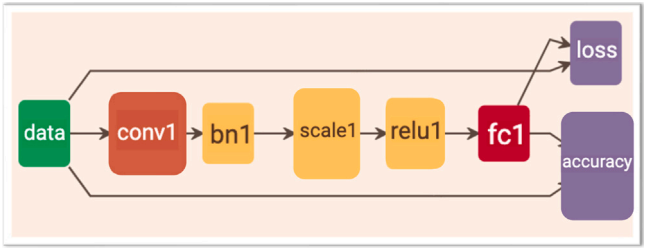


Fig. 6. Single layer Convolution-BatchNorm-Scale-Relu (CBSR) model structure.

We employed a sliding window technique to effectively reduce a continuous data stream into smaller, analyzable segments. We set the window size to precisely 170 data points, which corresponds to about 2.83 s at a 60 Hz sampling rate. To optimize the analysis, we implemented a 50% overlap between consecutive windows. This specific degree of overlap was critical for capturing essential transitional events, like the onset of a fall, while also keeping the data processing load manageable for computational efficiency.

For labeling purposes, we adhered to the principle of assigning labels based on the most dominant event in each data window. Consequently, a window that captures the beginning of a fall is labeled as a fall event. This approach ensures that the model is accurately trained to recognize and respond to critical moments that define each event.

4.3. CBSR neural network model

The *Convolution-Batch Normalization-Scaling-ReLU* (CBSR) structure, often employed in residual convolutional neural networks (CNNs), is a popular deep-learning neural network layer configuration. The architecture of the CBSR neural network is depicted in Fig. 6.

Each CBSR unit is composed of four key components: The convolutional layer functions as the feature extractor from the input data. It performs a convolution operation by convolving a filter with the input data, resulting in a generated feature map (Albawi, Mohammed, & Al-Zawi, 2017). The batch normalization layer then normalizes the data in each batch, aiming to bring the mean of the input data close to 0 and the standard deviation near 1. This normalization process speeds up the network's training and enhances the model's generalization ability (Ioffe & Szegedy, 2015). Following this, the scale layer introduces a modifiable scale parameter to fine-tune the output from the batch normalization layer, allowing the network to adjust the scaling of individual features, thus enhancing the model's expressiveness. Finally, the ReLU (Rectified Linear Unit) serves as an activation function, enabling the CBSR unit to learn intricate patterns in the data, crucial for distinguishing between fall and non-fall events. The input for the ReLU includes a learnable scale parameter from the batch normalization layer, further aiding the network in dynamically adjusting feature scaling for improved model expressiveness (Agarap, 2019).

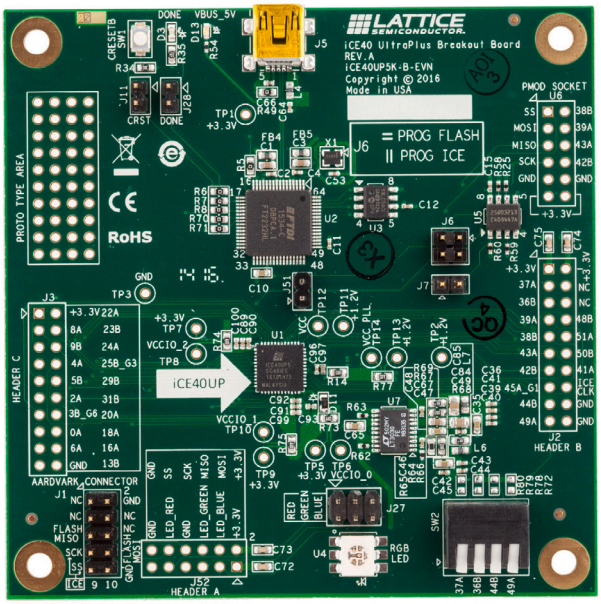


Fig. 7. Top view of the Lattice iCE40 UltraPlus Breakout Board. The actual ICE40UP5K FPGA chip is pointed to by the white arrow.

5. Design

After understanding the importance of a fall detection system (FDS), we came up with a SystemVerilog design where we combine an FPGA, sensor, and machine learning model. As an overall high-level description of this system, here the implementation is that an FPGA collects movement data from the IMU sensor using the Serial Peripheral Interface (SPI) protocol and stores data in the embedded block memory of the FPGA so that a machine learning model, which detects a fall, can access the data from a memory block to finally determine if that particular movement was a fall or not.

To validate the SystemVerilog model, the model was simulated with Lattice Radiant v3.1 and then connected to an STMicroelectronics LSM6DSOX iNEMO inertial module with a Lattice iCE40 UP5K FPGA. We used a Saleae logic analyzer to debug and verify SPI communication.

5.1. iCE40UP5K and LSM6DSOX

For our proposed FDS system, we have chosen the iCE40 UltraPlus FPGA which is manufactured by Lattice Semiconductor. This FPGA is claimed to be the world's smallest FPGA, which would be a major reason for us to select it, as we need a small wearable that allows for the implementation of an embedded neural network for classifying falls.

One main advantage of choosing the iCE40 Ultra Plus FPGA is that it is a so-called *edge intelligent* FPGA equipped with 5k lookup tables (LUTs). The iCE40 UltraPlus FPGA is capable of implementing neural networks for real-time pattern matching for continuous edge inferencing. The latency and cloud intelligence of the system can be optimized by designers to see a great reduction in power consumption along with reduced cost (Lattice Semiconductor, 2018a).

The LSM6DSOX iNEMO inertial module is a system-in-package that includes both a 3D digital gyroscope and an accelerometer. This inertial module performs a variety of measurements, including those of angular rate sensitivity, linear acceleration rate, angular rate, ultra-power modes, and acceleration RMS noise in low power mode (STMicroelectronics, 2019).

We have chosen to design an EBR with the write port address depth to be 34 with each port having a data width of 8 bits; the same is maintained with the read port as well. The output data from the LSM6DSOX sensor is saved in 34 distinct registers, each of which is an 8-bit register, hence EBRAM (Embedded Block RAM) is required in the FPGA iCE40UP to store data in the FPGA for later usage as input to an embedded neural network. For our design needs, we have selected a 34-bit embedded block RAM depth and an 8-bit address size. The FPGA receives data from the inertial sensor and stores it in EBRAM. When the last address in EBRAM is written, the FPGA implementation logic starts to overwrite the previous values at the beginning address with newly received data. It is a dual-port BRAM that can read data from, and write data to, memory at the same time, simultaneously. To analyze the output of BRAM using a logic analyzer, the output address is auto-incremented after every 1 s (Lattice Semiconductor, 2018a).

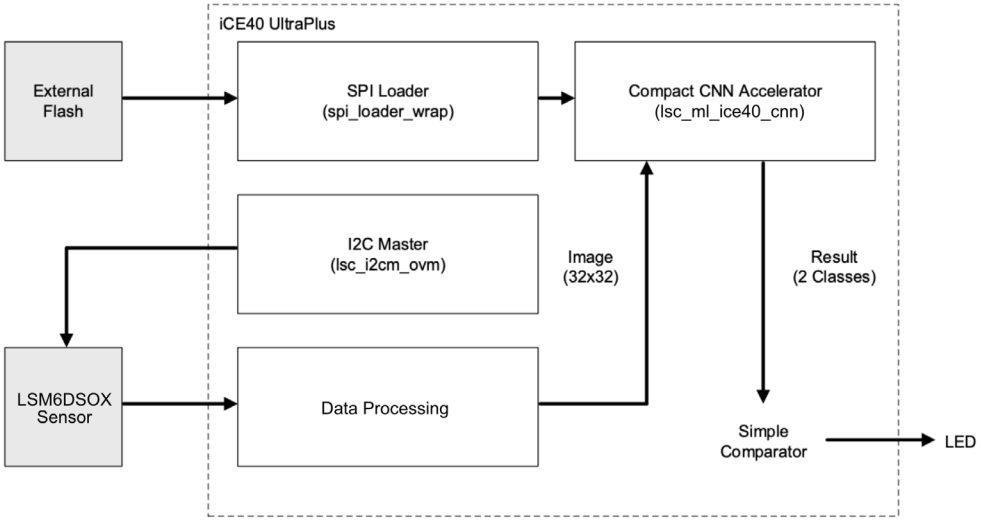


Fig. 8. Top Level Block Diagram for Fall Detection on the Lattice Semiconductor iCE40.

5.2. Circuit design

We have developed an innovative Fall Detection Sensor (FDS) utilizing an ultra-low power FPGA from Lattice Semiconductor. This FPGA is equipped to handle on-device machine learning convolutional architectures through the Lattice Compact CNN Accelerator Engine (Lattice Semiconductor, 2020). Our sensor design is intended to demonstrate the application of machine learning with the Lattice Compact CNN Accelerator Engine in fall detection.

The design of the fall detection system is outlined in a block diagram, which will be followed by detailed descriptions of each block module. As depicted in Fig. 8, the block diagram is divided into white and gray sections. The white blocks signify the main top-level module implemented on the Lattice iCE40 UltraPlus FPGA, and the gray blocks represent the external components. The dotted line in the diagram demarcates the boundary of the iCE40 UltraPlus FPGA. Within this setup, the Compact CNN Accelerator Engine processes inputs and produces two output values. These outputs are then analyzed to ascertain whether they indicate a fall event.

The Compact CNN Accelerator IP Core's interface is depicted in Fig. 9, showcasing the various ports available for the IP core. In our implementation, we utilize the ports *i_din*, *i_we*, *o_dout*, and *o_we*. This forms part of a CNN (convolutional neural network) module which is equipped with multiple input and output ports. The module is designed to receive a clock signal and a reset signal. In response, it generates a range of outputs associated with the neural network's data processing. These outputs include signals for reading and writing readiness, cycle counts, and output commands. The module is also capable of receiving input data, processing it through the neural network, and then outputting the processed data. Additionally, it features various debugging and status signals that monitor the neural network's functionality.

Using the Caffe framework (Jia et al., 2014), we compile a trained Convolutional, Batch Normalization, Scaling, and ReLU (CBSR) CNN model within the Lattice SensAI Neural Network Compiler Software (Lattice Semiconductor, 2018b). The Compiler Software converts the trained network into a fixed-point representation and produces a binary file. This binary file is then programmed onto a Lattice iCE40 UltraPlus Breakout Board, as illustrated in Fig. 7. The breakout board is utilized for prototyping our design. For our data set, we adopt an 80:20 training/testing division, using 80% of the collected fall data for training the model and the remaining 20% for testing its effectiveness.

The digital circuit diagram is shown in Fig. 10. This digital system can communicate between a sensor and a compact convolutional neural network (CNN) accelerator for processing data. The sensor interfaces with the system through the SPI communication protocol, utilizing a set of lines — MISO, MOSI, SCK, and CS — to transmit its data to the `spi_loader_single_spram` block, which is responsible for loading the sensor data into a single-port RAM. Once the data is in memory, the CNN accelerator can access it for processing. The CNN accelerator, optimized for performing the computationally intensive operations associated with neural networks, processes the sensor data according to its programmed algorithm. Upon completion, the processed data, which could be the result of an image or signal analysis, is sent to the `SerialTX_Top` block. This block then transmits the results externally via a UART interface, which a computer or another digital system could read for further use or analysis. Additionally, the system includes LED indicators driven by the `led_driver` block and employs SPI and the compact CNN engine for processing input data. These inputs are translated into binary values 0 or 1, triggering the onboard LED to switch between green and red states, signaling a no-fall or fall event, respectively.

Fig. 10 shows the schematic diagram of our proposed convolutional neural network (CNN) accelerator. At its core is the `compact_cnn` module, tasked with processing data received from the `iimg0` module. The `iimg0` module pre-processes the input before sending it to the CNN accelerator.

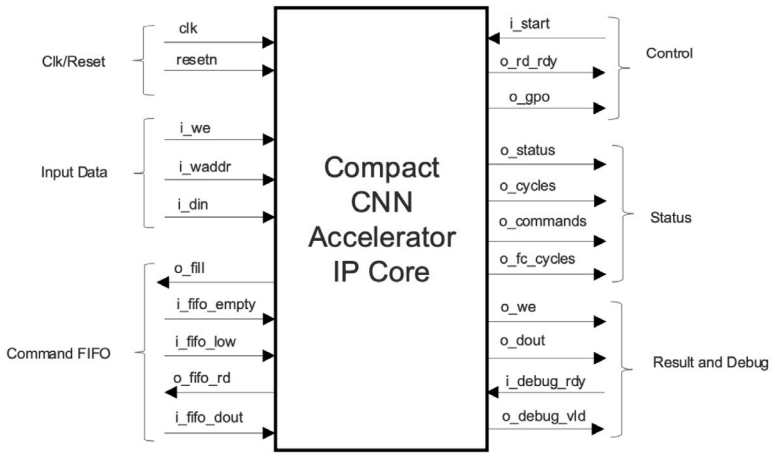


Fig. 9. Compact CNN Accelerator IP Core Interface Diagram.

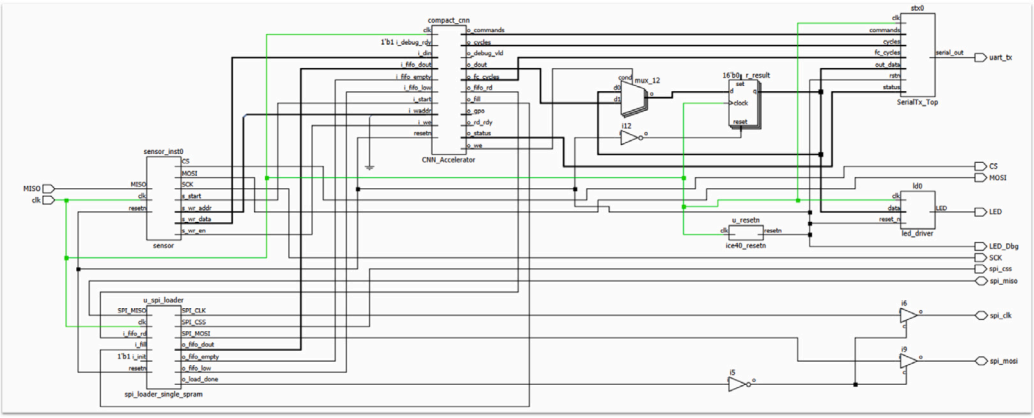


Fig. 10. Digital Circuit Diagram includes Input, SPI, Compact CNN Accelerator Engine, and Output Components.

The system includes the `SerialTx_Top` module for real-time communication and monitoring, enabling serialized data transmission to external systems. This is particularly useful for applications requiring immediate feedback. A key aspect of this design is the SPI interface (`spi_loader` & `spi_loader_single_spram`), which allows for quick data transfer and dynamic system configuration. The inclusion of Single Port RAM (SP_RAM) ensures data integrity and provides buffer storage for consistent data flow.

The *compact_cnn* is the linchpin of our design, optimized for low-latency and high-throughput operations. Unlike traditional, large, and power-intensive CNNs, the *compact_cnn* is specifically designed for embedded, real-time applications like fall detection. It utilizes fewer but more efficient convolution layers, maintaining essential feature extraction capabilities.

5.3. SPI communication

The flow chart shown in Fig. 11 describes the Verilog coding algorithm implemented in the Lattice FPGA for the configuration of the LSM6DSOX. The FPGA stores the yaw, pitch, roll, X, Y, and Z data in the FPGA's internal Embedded Block RAM (EBR) for pending input into a neural network input layer.

In the initial state, the CS and SCK signals are initialized to 0, after the positive edge of the clock the chip enable is high, and then we wait for 90us after which we set the chip enable to low and wait for 15us before sending data. After 15us, the chip enable is set to low and the SCLK signal becomes a 1.5 MHz clock signal. Now the MOSI address is configured to perform a read-and-write operation. When reading data from the sensor, MISO will receive the data on the SDO wire of the sensor. In the case where we are writing data to the sensor, MOSI will send out the data like we are receiving the accelerometer 3-axis reading address or orientation 3-axis reading. Once this process of read/write is done, we wait for 90us, and after that, we enter a loop of sending and receiving data from the sensor. This algorithm forms a Finite State Machine (FSM), where we can stop the communication link whenever we have completed sending and receiving data.

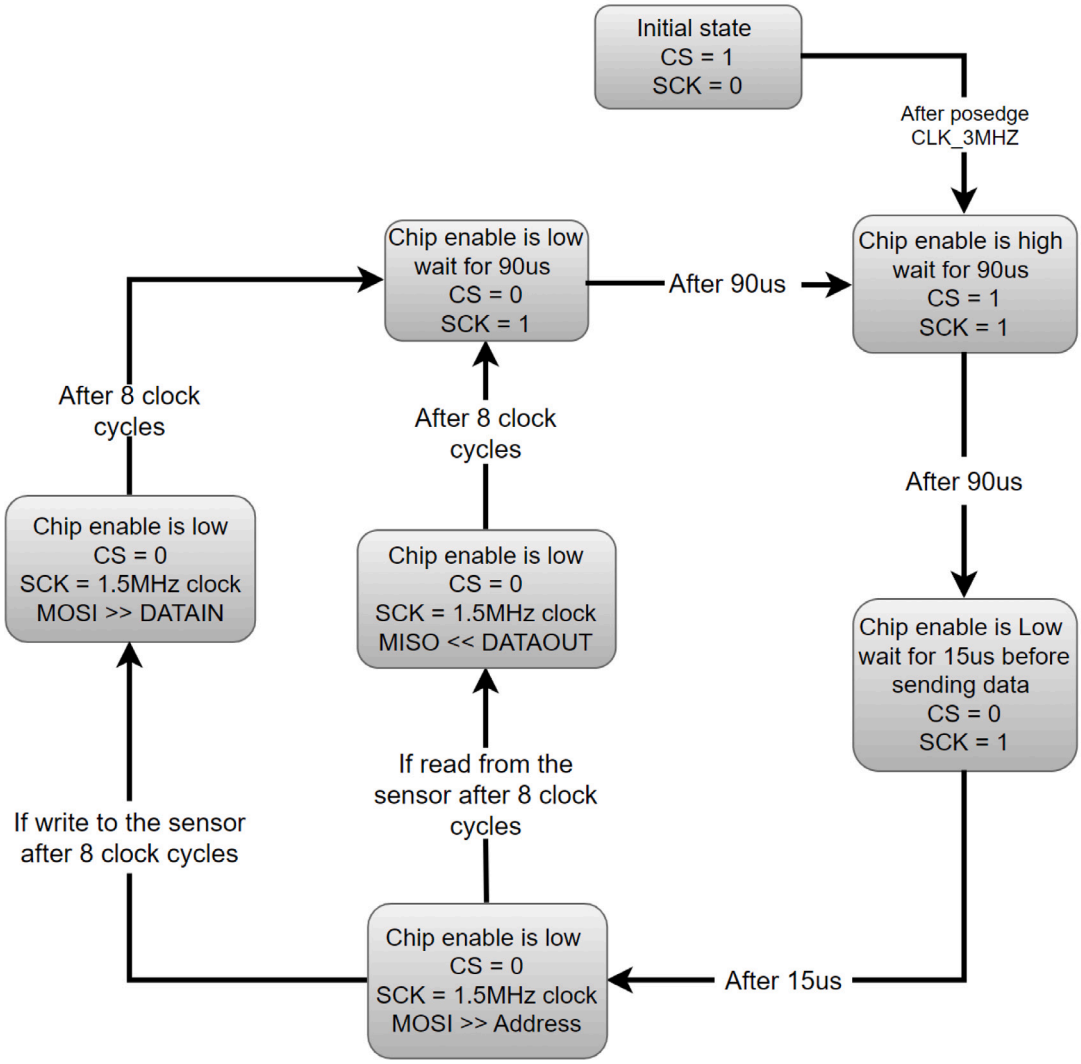


Fig. 11. Figure of timing for sending data to the sensor from the FPGA.

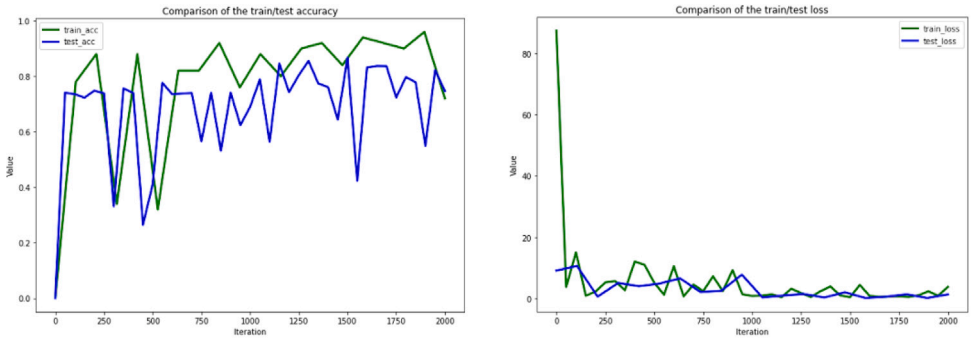
To instantiate a SPI connection between the LSM6DSOX iNEMO inertial module and the iCE40 UltraPlus FPGA, we need to configure specific control registers which are mentioned in the LSM6DSOX iNEMO inertial module datasheet (STMicroelectronics, 2019). The operating mode needs to be configured. The LSM6DSOX allows for varied ODRs (Output Data Rates) and power modes, as well as independent on/off switching of the accelerometer and gyroscope (STMicroelectronics, 2019).

There are three operating modes for the LSM6DSOX:

- Disabled gyroscope in power-down mode and accelerometer active.
- Power-down mode for the accelerometer and the gyroscope active.
- Independent ODR for the gyroscope and accelerometer sensors.

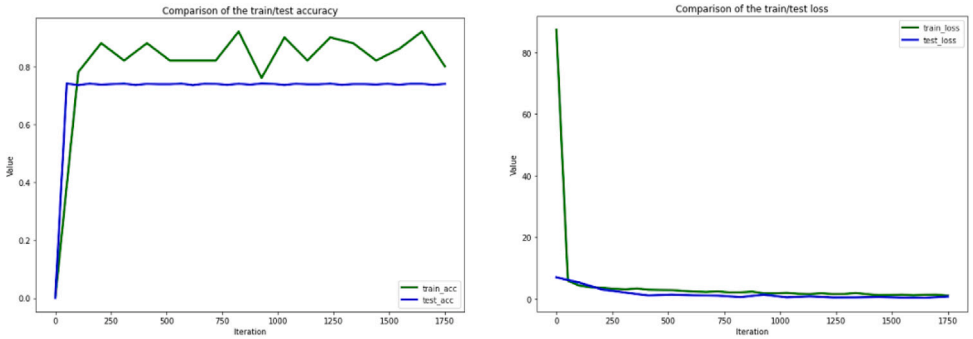
Writing ODR_XL[3:0] in CTRL1_XL (10h) will activate the accelerometer while writing ODR_G[3:0] in CTRL2_G (11h) will activate the gyroscope.

The FIFO_MODE_[2:0] bits in the FIFO_CTRL4 (0Ah) register are used to switch between each mode. When the FIFO counter hits a specified threshold, the flag COUNTER_BDR_IA in FIFO_STATUS2 (3Bh) warns the user CNT_BDR_TH_[10:0] field in COUNTER_BDR_REG1 (0Bh). This enables the reading of the FIFO to be triggered with the desired sensor latency. COUNTER_BDR_REG1 (0Bh) determines the upper limit for the batching event's internal counter. The COUNTER_BDR_IA flag in FIFO_STATUS2 (3Bh) is set to '1' when this counter hits the threshold, after which it is reset (STMicroelectronics, 2019).



(a) accuracy (b) loss

Fig. 12. Comparison of training and testing on a one-layer CBSR architecture.



(a) accuracy (b) loss

Fig. 13. Comparison of training and testing on three-layer CBSR architecture.

6. Results

6.1. Model training and Testing

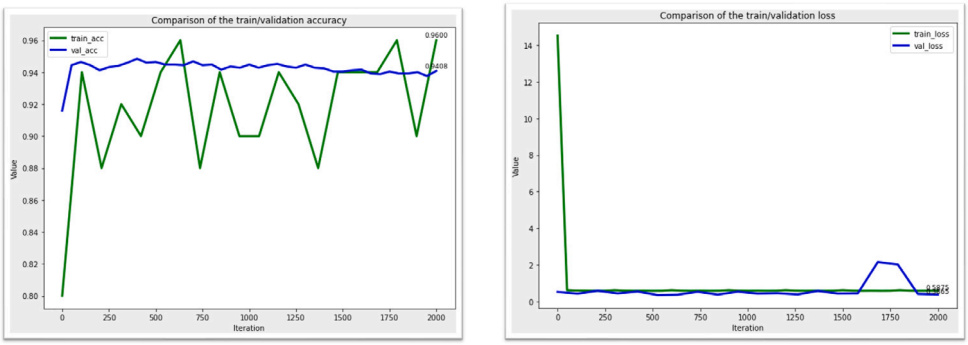
In our research, we made strategic adjustments to the structure of the Convolution-Batch Normalization-Scale-Relu (CBSR) model to identify its optimal configuration. We experimented by altering the original parameters of our CNN model, particularly by varying the number of CBSR layers. Our findings indicated that implementing a sequence of three consecutive CBSR layers, labeled as *CBSR-CBSR-CBSR*, achieved the highest testing accuracy, reaching 85%. However, it is crucial to note that overloading the model with convolutional layers can lead to complications. An error emerges when too many convolutional layers are added, causing the dimensions of the resulting matrix to reduce to 1×1. In such situations, adding additional convolutional layers becomes impractical. We have documented the performance in terms of accuracy and loss for a single CBSR layer in Fig. 12, and similarly, the results for three CBSR layers are presented in Fig. 13.

In our efforts to improve the model's accuracy, we refined the parameters of the CBSR model. By employing a solitary CBSR structure, we attained an impressive accuracy rate of 95%, as depicted in Fig. 15. The performance in terms of accuracy and loss for this single-layer CBSR architecture is detailed in Fig. 14. Additionally, the Confusion Matrix for this model is illustrated in Fig. 15, effectively demonstrating the correlation between the model's predictions and the actual (ground truth) labels. This CBSR Caffe model underwent training with a dataset comprising 500 instances of fall data and 2500 instances of no-fall data.

6.2. Power overview

The power matrix, as illustrated in Fig. 16 and sourced from Lattice Radiant Software, presents a detailed view of the current consumption for various electronic components under different power supplies, including V_{cc} and several V_{ccio} voltages (3.3 V, 2.5 V, and 1.8V). The components analyzed include Logic Blocks, Clocks, I/Os, PLLs, Block RAM, DSPs, SRAM, LEDs, and other miscellaneous elements.

In terms of current usage, the Logic Block registers a notable consumption of 22 mA from the V_{cc} supply. This highlights its significant role in logic operations and implies extensive use of logic gates, especially as part of the Compact CNN Accelerator



(a) accuracy (b) loss

Fig. 14. Learning curves of single CBSR unit architecture.

TARGET / OUTPUT	Predicted Fall	Predicted No Fall	SUM
Actual Fall	472 15.73%	28 0.93%	500 94.40% 5.60%
Actual No Fall	135 4.50%	2365 78.83%	2500 94.60% 5.40%
SUM	607 77.76% 22.24%	2393 98.83% 1.17%	2837 / 3000 94.57% 5.43%

Fig. 15. Software Confusion Matrix for CBSR machine learning model.

Supply\Blocks	Logic Block	Clocks	I/O	PLL	Block RAM	DSP	SRAM	LED	Misc	Total
Vcc	0.000027	0.00001	0.000039	0.000002	0.000008	0.000003	0.000013	0.000001	0.00001	0.000113
Vccio 3.3	0	0	0.000048	0	0	0	0	0	0	0.000049
Vccio 2.5	0	0	0	0	0	0	0	0	0	0
Vccio 1.8	0	0	0	0	0	0	0	0	0	0
Vccio 1.2	0	0	0	0	0	0	0	0	0	0
Total(Watt)	0.000027	0.00001	0.000087	0.000002	0.000008	0.000003	0.000013	0.000001	0.00001	0.000162

Fig. 16. Power Matrix.

within the block. The I/O Blocks show varying power usage across different supplies, with 0.032 mA drawn from V_{cc} and 0.015 mA from V_{ccio} 3.3 V. This variation in power draw could be due to the diverse roles of I/O pins, which may involve different signaling standards or operational modes that rely on specific voltage levels. The miscellaneous components, consuming 0.009 mA from the V_{cc} supply, suggest the inclusion of additional electronic parts or subsystems essential for secondary functions like signal conditioning, biasing, or other supportive tasks.

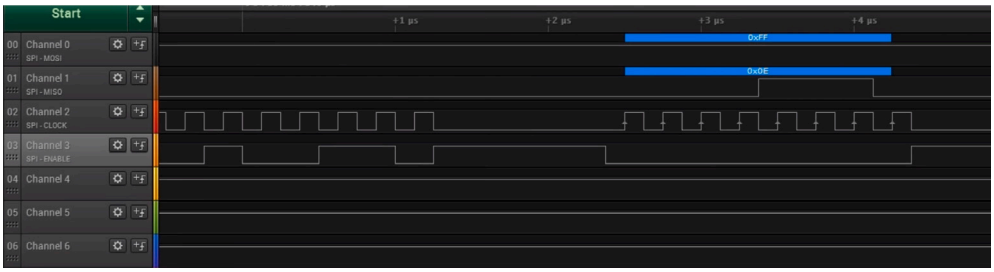


Fig. 17. SPI connection in Saleae Logic Analyzer figure.

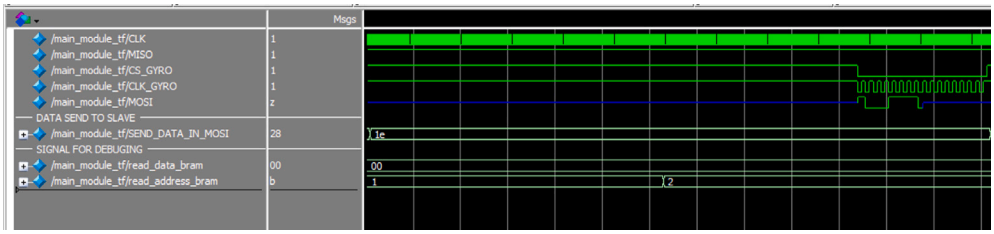


Fig. 18. Data 1Eh - STATUS_REG (1Eh) figure.

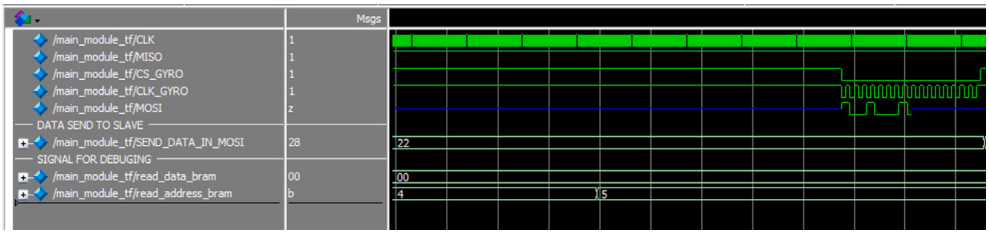


Fig. 19. Data 22h - OUTX_L_G (22h) figure.

The notably low current consumption by the Logic Block and I/O from the V_{cc} supply, which is likely a primary factor in dynamic power usage, can be attributed to the charging and discharging of internal capacitive devices during operation. This observation points to a critical area for power optimization, such as implementing techniques like clock gating or power gating, to effectively manage power usage during times of low or no activity in the Logic Block.

6.3. SPI connection

Results were obtained from the SPI connection between the LSM6DSOX iNEMO inertial module and the iCE40 Ultra from a Saleae Logic analyzer and Lattice Radiant simulator waveform plots. Data values are expressed as 16-bit words using two's complement representation. We connected a Saleae Logic Analyzer, shown in Fig. 17, to the FPGA in order to analyze the data sent and received over a SPI communication bus. The Saleae Logic Analyzer is connected to the FPGA and the iNEMO inertial module. We can see MISO, MOSI, clock, and enable signals. We show that a connection is made, and the actual data is being received by the data registers of the FPGA.

The data is sent from the Lattice Semiconductor iCE40 UltraPlus FPGA to the ST Microelectronics LSM6DSOX to enable the iNEMO inertial module to send back the required data to the iCE40 UltraPlus FPGA. We first set the *STATUS_REG* of the iNEMO inertial module by sending 1Eh 18 to this register, to enable the SPI interface to get the acceleration and angular rate data from the iNEMO inertial module.

To receive the angular rate data concerning the x-axis represented in a 16-bit word in two's complement, we need to send 22h 19 and 23h 20 to obtain the Least Significant Byte (LSB) and the Most Significant Byte (MSB), respectively, as seen in Figs. 19 and 20 on the line titled *SEND_DATA_IN_MOSI*.

6.4. Real-world hardware results

A test was conducted on our CBSR CNN model using a Lattice iCE40UP5K board. For this test, we manually fed eighty-five matrices, each of size 32 × 32, into the input layer of the model through Embedded Block Ram (EBR) over an SPI bus. The model's

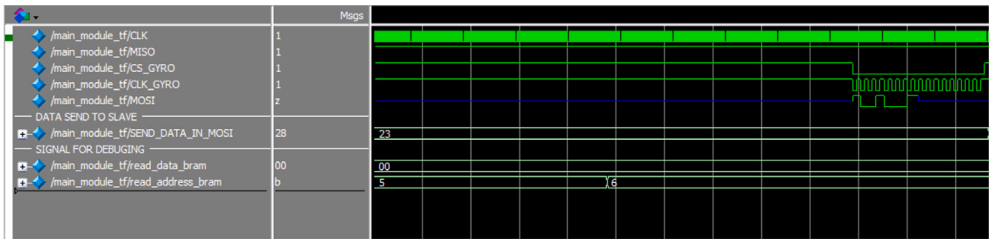


Fig. 20. Data 23h - OUTX_H_G (23h) figure.

TARGET / OUTPUT	Predicted Fall	Predicted No Fall	SUM
Actual Fall	29 34.12%	8 9.41%	37 78.38% 21.62%
Actual No Fall	6 7.06%	42 49.41%	48 87.50% 12.50%
SUM	35 82.86% 17.14%	50 84.00% 16.00%	71 / 85 83.53% 16.47%

Fig. 21. Hardware Confusion Matrix for CBSR machine learning model.

output was monitored by observing changes in the color of an LED. The Confusion Matrix, representing the performance of this model when implemented on the iCE40UP5K board, is displayed in Fig. 21.

We initiated a real-world evaluation of our innovative sensor system, which incorporates the iCE40UP5K integrated circuit and the LSM6DSOX IMU, all encased within a custom-designed 3D-printed housing. A key feature of this system is its response to fall detection: upon recognizing a fall, the iCE40UP5K's LED indicator is programmed to blink red for three seconds. Simultaneously, we capture critical data through our data logger, which utilizes the LSM6DSOX IMU and is equipped with a real-time clock for precise time-stamping. Our system operates at a sampling rate of about 500 samples per second, requiring 170 samples to form a matrix for analysis. This efficiency allows the system to make approximately three fall-inferencing decisions per second.

This comprehensive test of our CBSR CNN model was implemented on a Lattice iCE40UP5K board, housed in a custom 3D-printed case designed specifically for this purpose. This case also accommodates the LSM6DSOX IMU. We undertook a more realistic approach by performing 60 actual human subjects' fall tests with a participant. During these tests, data was fed into the model's input layer through Embedded Block RAM (EBR) over an SPI bus, reflecting real-world scenarios of falls.

The output of the model was carefully observed by monitoring the LED on the iCE40UP5K board, which was programmed to change color upon fall detection. The performance of this live test, including the model's response to actual falls, is detailed in the Confusion Matrix shown in Fig. 22. This practical approach not only tested the model's accuracy in detecting falls, but also showcased the effectiveness of the integrated system, comprising the iCE40UP5K board and LSM6DSOX IMU, in a real-world setting.

Our evaluation of the fall detection system unfolded in stages, starting with a software accuracy of 94.5% on a GPU server, indicating strong potential in high-computational settings. Hardware testing on an FPGA board revealed a lower accuracy of 83.5%, highlighting the transition challenges to physical implementation. Real-world application, detailed in Fig. 22, involved the model on FPGA with live sensor data, aiming to replicate operational conditions and assess practical performance. These phases, from software simulations to real-world deployment, highlighted the system's testing across different environments for robust and reliable operational performance.

Despite the high accuracy achieved by our fall detection system, we observed specific scenarios where performance was suboptimal. These failure cases primarily involved atypical fall patterns not well-represented in our training data and instances of sensor misplacement, leading to inaccurate data capture. Such conditions occasionally resulted in false negatives, underscoring the necessity for expanding our dataset to cover a broader spectrum of fall types and enhancing the system's adaptability to sensor positioning.

TARGET / OUTPUT	Predicted Fall	Predicted No Fall	SUM
Actual Fall	20 33.33%	5 8.33%	25 80.00% 20.00%
Actual No Fall	3 5.00%	32 53.33%	35 91.43% 8.57%
SUM	23 86.96% 13.04%	37 86.49% 13.51%	52 / 60 86.67% 13.33%

Fig. 22. Real-world Confusion Matrix for CBSR machine learning model.

7. Conclusion

We have developed an innovative fall detection system (FDS) device featuring an in situ machine-learning architecture, designed to determine whether a wearer has fallen. This device is powered by a low-energy Lattice Semiconductor iCE40UP5K FPGA, integrated with three-unit CBSR (Convolution-Batch Normalization-Scale-Relu) architecture, and is equipped with an STMicroelectronics LSM6DSOX IMU sensor. We successfully demonstrated a novel approach by employing a compact Convolutional Neural Network (CNN) core to expedite convolutional operations, such as inner products and multiply-add operations, on a shallow learning model designed for implementation on an ultra-low power FPGA.

The core focus of our research has been to create a reliable and efficient Fall Detection System (FDS). This involved the detailed development and testing of a CBSR model, which we have successfully implemented and validated in both software and hardware contexts, achieving testing accuracies of 94.5% and 83.5%, respectively. Our design, centered around the three-unit CBSR architecture within the iCE40UP5K FPGA, has proven effective for in-device, real-time processing.

Our system offers notable benefits, including user privacy and immediate response capability, making it highly suitable for practical applications where prompt fall detection and response are crucial. The strategic placement of the sensor on the shinbone, based on empirical data, has been key in capturing the intricate dynamics of falls, thus enhancing the accuracy and reliability of the FDS. Also, convenient for elderly persons, since the time required for a recharge is reduced.

While our discussion on existing fall detection algorithms was concise, reflecting our specific aim to validate the use of machine learning models on FPGAs, we acknowledge the rich landscape of research in fall detection techniques. These range from simple threshold-based methods to complex machine-learning algorithms applied directly to sensor data. Our future work will extend this discussion, comparing our FPGA-based CNN approach with other algorithms to elucidate its novelty, advantages, and the specific challenges it addresses within the realm of fall detection.

Nevertheless, we understand that refining the FDS is an ongoing, multifaceted endeavor. Despite the encouraging outcomes, we face challenges such as data imbalance and limitations in dataset size. These issues highlight the need for continued efforts in model improvement and adaptation to diverse real-world environments.

Our focus in this study was on demonstrating the application of image-based CNNs on FPGAs for fall detection, involving the transformation of 1D sensor data into 2D images. A direct comparison with 1D CNN approaches like TCN, although valuable, was outside our immediate scope. Future work will explore this comparison, emphasizing key metrics such as accuracy and efficiency, to enhance our methodology for real-time fall detection.

In addition to our research and development efforts, we have conducted several real-world tests to validate our system's performance with an 86.6% accuracy. These tests have not only demonstrated the effectiveness of our design in practical scenarios but also confirmed the reliability of our results, further solidifying the merit of our FDS in real-life applications.

In our future study, we will pursue the implementation of hardware-accelerated temporal Convolutional Neural Network architectures, Graph Convolutional Neural Networks, and their binarized variants. We will perform comparative analysis with other state-of-the-art research algorithms through the same data set and hardware platform, to prove that our model is well-motivated and benchmarked.

CRediT authorship contribution statement

Jingxiao Tian: Writing – original draft, Visualization, Validation, Software, Methodology. **Patrick Mercier:** Writing – review & editing, Conceptualization. **Christopher Paolini:** Writing – review & editing, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

Research reported in this publication/abstract was supported by the National Institute on Minority Health and Health Disparities of the National Institutes of Health under Award Numbers S21MD010690 (SDSU HealthLINK Endowment) and U54MD012397 (SDSU HealthLINK Center). The content is solely the responsibility of the authors and does not necessarily represent the official views of the National Institutes of Health.

References

Agarap, A. F. (2019). Deep learning using rectified linear units (ReLU). arXiv:1803.08375.

Albawi, S., Mohammed, T. A., & Al-Zawi, S. (2017). Understanding of a convolutional neural network. In *2017 international conference on engineering and technology* (pp. 1–6). http://dx.doi.org/10.1109/ICEngTechnol.2017.8308186.

Bagala, F., Becker, C., Cappello, A., Chiari, L., Aminian, K., Hausdorff, J. M., et al. (2012). Evaluation of accelerometer-based fall detection algorithms on real-world falls. *PLoS One*, [ISSN: 1932-6203] *7*(5), Article e37062. http://dx.doi.org/10.1371/journal.pone.0037062.

Bet, P., Castro, P. C., & Ponti, M. A. (2019). Fall detection and fall risk assessment in older person using wearable sensors: A systematic review. *International Journal of Medical Informatics*, *130*, Article 103946.

Boutellaa, E., Kerdjidj, O., & Ghanem, K. (2019). Covariance matrix based fall detection from multiple wearable sensors. *Journal of Biomedical Informatics*, [ISSN: 1532-0464] *94*, Article 103189. http://dx.doi.org/10.1016/j.jbi.2019.103189, URL https://www.sciencedirect.com/science/article/pii/S1532046419301078.

Burns, E., & Kakara, R. (2018a). Deaths from falls among persons aged ≥ 65 years — United States, 2007–2016. *MMWR. Morbidity and Mortality Weekly Report*, *67*, 509–514. http://dx.doi.org/10.15585/mmwr.mm6718a1.

Burns, E., & Kakara, R. (2018b). Deaths from falls among persons aged ≥ 65 years - United States, 2007–2016. *MMWR. Morbidity and Mortality Weekly Report*, *67*(18), 509–514. http://dx.doi.org/10.15585/mmwr.mm6718a1, Published May 11, 2018. URL https://www.cdc.gov/mmwr/volumes/67/wr/mm6718a1.htm.

Carreira, J., & Zisserman, A. (2017). Quo vadis, action recognition? A new model and the kinetics dataset. In *2017 IEEE conference on computer vision and pattern recognition* (pp. 4724–4733). http://dx.doi.org/10.1109/CVPR.2017.502.

Casilari, Alvarez-Marco, & García-Lagos (2020). A study of the use of gyroscope measurements in wearable fall detection systems. *Symmetry*, 649.

Casilari, E., Santoyo-Ramón, J. A., & Cano-García, J. M. (2017). UMAFall: A multisensor dataset for the research on automatic fall detection. *Procedia Computer Science*, [ISSN: 1877-0509] *110*, 32–39. http://dx.doi.org/10.1016/j.procs.2017.06.110, URL http://www.sciencedirect.com/science/article/pii/S1877050917312899.

Chander, H., Burch, R. F., Talegaonkar, P., Saucier, D., Luczak, T., Ball, J. E., et al. (2020). Wearable stretch sensors for human movement monitoring and fall detection in ergonomics. *International Journal of Environmental Reserach and Public Health*, *17*(10).

Cippitelli, E., Fioranelli, F., Gambi, E., & Spinsante, S. (2017). Radar and RGB-depth sensors for fall detection: A review. *IEEE Sensors Journal*, *17*(12), 3585–3604. http://dx.doi.org/10.1109/JSEN.2017.2697077.

Diaz, A., Prado, M., Roa, L., Reina-Tosina, J., & Sanchez, G. (2004). Preliminary evaluation of a full-time falling monitor for the elderly. In *The 26th annual international conference of the IEEE engineering in medicine and biology society*: *vol. 1*, (pp. 2180–2183). http://dx.doi.org/10.1109/IEMBS.2004.1403637.

Feichtenhofer, C., Fan, H., Malik, J., & He, K. (2019). SlowFast networks for video recognition. In *2019 IEEE/CVF international conference on computer vision* (pp. 6201–6210). http://dx.doi.org/10.1109/ICCV.2019.00630.

Grossman, D. C., Curry, S. J., Owens, D. K., Barry, M. J., Caughey, A. B., Davidson, K. W., et al. (2018). Interventions to prevent falls in community-dwelling older adults: US preventive services task force recommendation statement. *JAMA*, *319*(16), 1696–1704. http://dx.doi.org/10.1001/jama.2018.309.

Gurley, R. J., Lum, N., Sande, M., Lo, B., & Katz, M. H. (1996). Persons found in their homes helpless or dead. *New England Journal of Medicine*, [ISSN: 0028-4793] *334*(26), 1710–1716. http://dx.doi.org/10.1056/NEJM199606273342606.

Hussain, F., Hussain, F., Ehatisham-ul Haq, M., & Azam, M. A. (2019). Activity-aware fall detection and recognition based on wearable sensors. *IEEE Sensors Journal*, *19*(12), 4528–4536. http://dx.doi.org/10.1109/JSEN.2019.2898891.

Hwang, J., Kang, J., Jang, Y., & Kim, H. (2004). Development of novel algorithm and real-time monitoring ambulatory system using bluetooth module for fall detection in the elderly. In *The 26th annual international conference of the IEEE engineering in medicine and biology society*: *vol. 1*, (pp. 2204–2207). http://dx.doi.org/10.1109/IEMBS.2004.1403643.

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In F. Bach, & D. Blei (Eds.), *Proceedings of machine learning research*: *vol. 37*, *Proceedings of the 32nd international conference on machine learning* (pp. 448–456). Lille, France: PMLR, URL https://proceedings.mlr.press/v37/ioffe15.html.

Jia, Y., Shelhamer, E., Donahue, J., Karayev, S., Long, J., Girshick, R., et al. (2014). Caffe: Convolutional architecture for fast feature embedding. arXiv preprint arXiv:1408.5093.

Komeylian, S. (2021). Deep neural network modeling of different antenna arrays; analysis, evaluation, and application. *IEEE Canadian Journal of Electrical and Computer Engineering*, *44*(3), 261–274.

Komeylian, S., Paolini, C., & Sarkar, M. (2023). Overcoming an evasion attack on a cnn model in the mimo-ofdm wireless communication channel. In *Advances in distributed computing and machine learning: proceedings of ICADCMl 2023* (pp. 71–88). Springer.

Lattice Semiconductor (2018a). URL http://www.latticesemi.com/Products/DesignSoftwareAndIP/IntellectualProperty/IPCore/IPCores04/CNN.

Lattice Semiconductor (2018b). *Lattice SensAI Neural Network Compiler Software User Guide*. 5555 NE Moore Ct, Hillsboro, OR 97124: Lattice Semiconductor.

Lattice Semiconductor (2020). *Compact CNN Accelerator IP Core User Guide* (02038-1.7). 5555 NE Moore Ct, Hillsboro, OR 97124: Lattice Semiconductor.

Lindemann, U., Hock, A., Stuber, M., Keck, W., & Becker, C. (2005). Evaluation of a fall detector based on accelerometers: A pilot study. *Medical & Biological Engineering & Computing*, [ISSN: 1741-0444] *43*(5), 548–551. http://dx.doi.org/10.1007/BF02351026.

Maglogiannis, I., Ioannou, C., & Tsanakas, P. (2016). Fall detection and activity identification using wearable and hand-held devices. *Integrated Computer-Aided Engineering*, *23*, 161–172.

Mobile help 2019. URL https://buy.mobilehelp.com/.

Musci, M., De Martini, D., Blago, N., Facchinetti, T., & Piastra, M. (2021). Online fall detection using recurrent neural networks on smart wearable devices. *IEEE Transactions on Emerging Topics in Computing*, *9*(3), 1276–1289. http://dx.doi.org/10.1109/TETC.2020.3027454.

Noury, N., Barralon, P., Virone, G., Boissy, P., Hamel, M., & Rumeau, P. (2003). A smart sensor based on rules and its evaluation in daily routines. In *Proceedings of the 25th annual international conference of the IEEE engineering in medicine and biology society (IEEE cat. no.03CH37439)*: *vol. 4*, (pp. 3286–3289). http://dx.doi.org/10.1109/IEMBS.2003.1280846.

Ojetola, O., Gaura, E., & Brusey, J. (2011). Fall detection with wearable sensors–safe (smart fall detection). In *Seventh international conference on intelligent environments IEEE* (pp. 318–321).

Paolini, C., Soselia, D., Baweja, H., & Sarkar, M. (2019). Optimal location for fall detection edge inferencing. In *2019 IEEE global communications conference* (pp. 1–6). http://dx.doi.org/10.1109/GLOBECOM38437.2019.9014212.

Porter, E. J. (2005). Wearing and using personal emergency response system buttons. *Journal of Gerontological Nursing*, *31*(10).

Ramachandran, A., & Karuppiah, A. (2020). A survey on recent advances in wearable fall detection systems. *BioMed Research International*, [ISSN: 2314-6133] *2020*, Article 2167160. http://dx.doi.org/10.1155/2020/2167160.

Santiago, J., Cotto, E., Jaimes, L. G., & Vergara-Laurens, I. (2017). Fall detection system for the elderly. In *2017 IEEE 7th annual computing and communication workshop and conference* (pp. 1–4). http://dx.doi.org/10.1109/CCWC.2017.7868363.

STMicroelectronics (2019). *iNEMO inertial module: always-on 3D accelerometer and 3D gyroscope*. (DS12814 - Rev 3), STMicroelectronics Inc., URL https://www.st.com/resource/en/datasheet/lsm6dsox.pdf.

Tian, Y., Che, Z., Bao, W., Zhai, G., & Gao, Z. (2020). Self-supervised motion representation via scattering local motion cues. In *Computer vision–ECCV 2020: 16th European conference, glasgow, UK, August 23–28, 2020, proceedings, part XIV 16* (pp. 71–89). Springer.

Tian, J., & Paolini, C. (2023). SDSU healthlink center falls dataset. http://dx.doi.org/10.17605/OSF.IO/WVEA3, URL https://osf.io/wvea3.

Tian, Y., Yan, Y., Zhai, G., Guo, G., & Gao, Z. (2022). EAN: Event adaptive network for enhanced action recognition. arXiv:2107.10771.

Torti, E., Fontanella, A., Musci, M., Blago, N., Pau, D., Leporati, F., et al. (2018). Embedded real-time fall detection with deep learning on wearable devices. In *2018 21st euromicro conference on digital system design* (pp. 405–412). http://dx.doi.org/10.1109/DSD.2018.00075.

Waheed, M., Afzal, H., & Mehmood, K. (2021). NT-FDS—A noise tolerant fall detection system using deep learning on wearable devices. *Sensors*, [ISSN: 1424-8220] *21*(6), http://dx.doi.org/10.3390/s21062006, URL https://www.mdpi.com/1424-8220/21/6/2006.

Yhdego, H., Li, J., Morrison, S., Audette, M., Paolini, C., Sarkar, M., et al. (2019). Towards musculoskeletal simulation-aware fall injury mitigation: Transfer learning with deep CNN for fall detection. In *2019 spring simulation conference* (pp. 1–12). http://dx.doi.org/10.23919/SpringSim.2019.8732857.